

Leveraging Cloud Native Design Patterns for Security-as-a-Service Applications

Kennedy A Torkura, Muhammad I.H Sukmana, Feng Cheng, and Christoph Meinel
 Internet Technologies and Systems Group
 Hasso Plattner Institute, University of Potsdam
 Potsdam, Germany

Email: {kennedy.torkura, muhammad.sukmana, feng.cheng, christoph.meinel}@hpi.de

Abstract—This paper discusses a new approach for designing and deploying Security-as-a-Service (SecaaS) applications using cloud native design patterns. Current SecaaS approaches do not efficiently handle the increasing threats to computer systems and applications. For example, requests for security assessments drastically increase after a high-risk security vulnerability is disclosed. In such scenarios, SecaaS applications are unable to dynamically scale to serve requests. A root cause of this challenge is employment of architectures not specifically fitted to cloud environments. Cloud native design patterns resolve this challenge by enabling certain properties e.g. massive scalability and resiliency via the combination of microservice patterns and cloud-focused design patterns. However adopting these patterns is a complex process, during which several security issues are introduced. In this work, we investigate these security issues, we redesign and deploy a monolithic SecaaS application using cloud native design patterns while considering appropriate, layered security counter-measures i.e. at the application and cloud networking layer. Our prototype implementation out-performs traditional, monolithic applications with an average *Scanner Time* of 6 minutes, without compromising security. Our approach can be employed for designing secure, scalable and performant SecaaS applications that effectively handle unexpected increase in security assessment requests.

Index Terms—Cloud-Security, Security-as-a-Service, Vulnerability Assessment, Cloud Native Applications

I. INTRODUCTION

The current rise in cyber-attacks correlates with the increasing number of vulnerability disclosures [1]. According to IBM X-Force Threat Intelligence Report 2017, the number of reported vulnerabilities in 2016 rose to an all-time highest of 10,197 vulnerabilities [2]. By implication, computer systems could be vulnerable to more malicious attacks if adequate measures are not implemented. Security assessment mechanisms are suitable for mitigating the above mentioned security challenge. These mechanisms are employed to identify and resolve vulnerabilities in computer systems and applications e.g. the number of vulnerabilities in applications could be reduced by integrating security assessments techniques in Continuous Development (CD) pipelines (see Figure 1). This is even more important now given the rapid rates of production cycles [3] i.e. faster *time-to-market*. Similarly, security assessments are imperatives for security in production environments via continuous security assessment schemes[4], which continuously evaluate computer systems for security vulnerabilities and mis-configurations. However, employing security assessments

could be difficult, requiring proficient skills and experience in computer security. Current security assessment approaches are manual, requiring complex security configuration hence unintuitive for end users. There is therefore the need to develop better security assessment approaches that are automated, pluggable and scalable.

SecaaS is a prospective approach that satisfies the above mentioned requirements. SecaaS follows after the Software-as-a-Service (SaaS) model by leveraging the cloud to provide security services. Hence, the services are self-managed, automated and scalable. However, the current approaches for deploying cloud applications such as SaaS and SecaaS are not optimal [5]. These applications do not maximally explore the cloud's potentials e.g. in terms of auto-scaling, elasticity and resiliency. Alternate approaches for designing and deploying applications that natively integrate into the cloud are required.

Cloud native design patterns recently appeared as a prospective approach to cloud application design, development and deployment [6]. These design patterns combine microservices with cloud focused design patterns to produce Cloud Native Applications (CNA). The success of companies such as Amazon, Uber and Netflix, is generally attributed to their deployment of CNA [7]. We believe that applying these patterns to SecaaS applications could improve security services delivery. But adopting cloud native design patterns is a complex procedure, requiring detailed planning and there are no well defined steps [8]. Moreover, security concerns are introduced during the migration process, yet these security issues are not being investigated [9]. For example, migrated applications automatically inherit enlarged attack surfaces inherent in microservices and are vulnerable to Man-in-the-Middle (MiTM) attacks and privacy intrusions by *honest but curious* cloud providers.

Contributions Our major contribution is the application of cloud native design patterns, a novel design pattern to SecaaS, thereby gaining several advantages e.g. better scalability. Furthermore, we investigate the security implications of adopting these design patterns and identify the security requirements. Consequently, we implement appropriate countermeasures from the cloud network perspective and the application layer. We aim at providing guidelines for future CNA adopters by answering the following questions:

- What are the security requirements for migrating mono-

lithic applications to cloud-native environments?

- How can SecaaS applications leverage cloud native design patterns to deliver scalable, secure and reliable services?

In the next section, we discuss related works, followed by an overview of CNA and a motivating example of deploying SecaaS in a Continuous Integration (CI) pipeline. In Section III, we highlight on security implications of CNA migrations, justify our adoption of cloud native design patterns and provide a description of our system model. The implementation details of our prototype are presented in Section V. In Section VI, we evaluate our work and conclude the paper in Section VII.

II. RELATED WORK

Few studies exist on cloud native design patterns and the security challenges of employing these patterns. Available literature consists majorly of migration experiences and lessons learnt. In [10], the authors investigated SecaaS and proposed a reference meta-model for modelling and analyzing design choices for SecaaS. Salah et al [11] proposed a SecaaS architecture, majorly comprised of commercial security products e.g. anti-virus software and security scanners. They validated the viability of providing cloud-based security with existing security solutions. The *FlowTap* primitive was proposed in [12] as a SecaaS for microservices deployed in the cloud. *FlowTap* augments existing cloud security monitoring services, and provides an approach for monitoring network traffic. Vijay et. al. presented a SecaaS model in [13], focusing on hypervisor security for CSPs. Their model also included security monitoring of tenant resources such as VMs, however the problem of application security was not tackled.

Existing experience reports [14][15][5] on migration of monolithic applications to CNA demonstrate use of microservices and cloud-focused design patterns to migrate applications to the cloud. However, the security challenges are not considered, as observed by Kratzke et. al's systemic mapping study and analysis of CNA[7]. The study investigated on CNA concepts and open research questions, and observed the need for security biased investigations. More recently, Leymann et al presented a detailed analysis of CNA including how they differ from traditional virtualization approaches [16][17]. The authors recommended CNA deployments on Platform as a Service (PaaS) clouds as a mitigation plan for security management since PaaS providers take responsibility for security tasks e.g. vulnerability patching. We focus on the security challenges of deploying CNA on Infrastructure as a Service (IaaS) platforms.

The major difference between our work and existing works is two fold. Firstly, in the context of SecaaS, we apply cloud native design patterns thereby gaining from its benefits such as scalability and resiliency. Secondly, we identify and mitigate the security challenges of migration to CNA. To the best of our knowledge, our work is the first tackling these challenges concurrently.

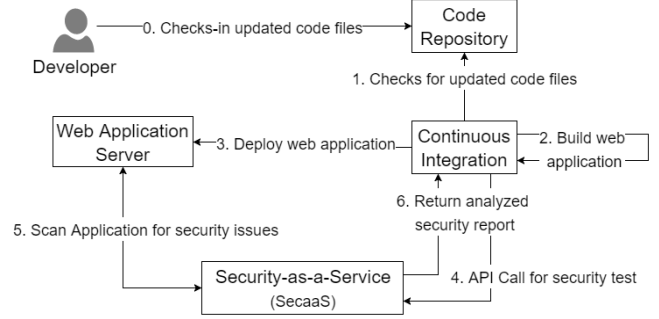


Figure 1: A Typical Deployment of cloud-native SecaaS in a Continuous Integration Pipeline

III. APPLYING CLOUD-NATIVE DESIGN PATTERNS TO SECURITY-AS-A-SERVICE APPLICATIONS

This section discusses the general concepts underlying cloud native design patterns and provides a motivating use-case of integrating a cloud-native SecaaS in a CI pipeline.

A. Cloud-Native Architectures - Design Principles and Motivations for Migration

Deployment of applications to cloud platforms is generally motivated by the advantages offered by the cloud. However, most deployments employ techniques specifically designed for traditional datacenters [16]. A typical example is the technique of bundling complete application stacks e.g. a three-tier web application into a cloud VM. The application becomes inflexible in terms of scalability, vertical scaling is possible but limited by some factors e.g the VM's RAM size. Alternately, the application can scale horizontally, though this is expensive since it entails VM replication for every additional instance. While these approaches have been commonly employed, new approaches used by companies e.g. Netflix are more efficient [18]. These new approaches, collectively referred to as cloud native design patterns, have been motivated by the desire for speedy innovation, highly available services, massive scalability and client diversity.

Cloud native design patterns aim at engineering applications to fully maximize the benefits of the cloud. Since cloud environments have specific characteristics, considering these characteristics during application development and deployment is imperative. For example, failures are common in cloud environments owing to the complexities of distributed systems, hence an application's ability to identify and handle failures improves resiliency. This is achieved by implementing the *circuit breaker* pattern.¹ Cloud native design patterns are still evolving, therefore a complete description of the underlying concepts is beyond this paper's scope. However, we point to references in academic and industrial domains. According to Fehling et. al[19] and Kratzke et al [8], cloud native design patterns include *resiliency*, *scalability*, *isolated states*, *elasticity* and *loose-coupleness*.

¹ <https://martinfowler.com/bliki/CircuitBreaker.html>

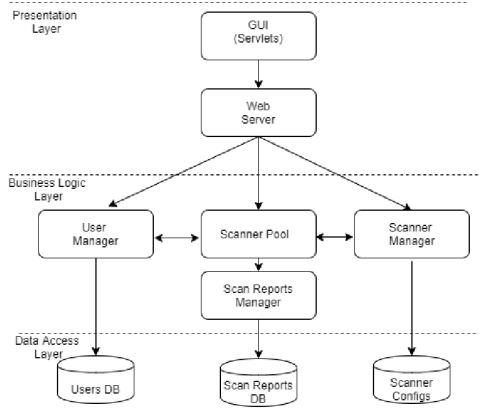


Figure 2: Architectural Diagram of the Monolithic CAVAS to be Migrated to CNA

Another popular description of the characteristics of CNAs is provided in *Twelve-factor applications*, a construct of the Heroku cloud engineering team.² These factors include *microservices*, *self-service agile infrastructures*, *APIs based collaboration* and *anti-fragility* [18]. In Section V, we provide practical details on how we implemented these properties in our prototype.

B. Use Case of a Cloud Native SecaaS - Security Integration in DevOps

The *DevOps* philosophy aims at increasing collaboration between operations and development teams in enterprises [20]. This implies better communication and joint decision-making between these teams, ultimately DevOps creates a viable environment for faster software release cycles. However, security teams are often sidelined, which negatively affects application security, e.g. introduction of security vulnerabilities[21]. Hence, deliberate efforts to integrate security testing into CI and/or CD i.e. *DevSecOps* are imperative for balancing speed and security. A possible approach consists in training DevOps on security roles. But this approach is not optimal and may be limited given the requirement for security expertise. Thus, another approach consists in employing security practitioners to directly collaborate with DevOps. A shortcoming with this approach is additional costs, particularly for small companies e.g. startups. Yet, another option is outsourcing security to SecaaS providers. SecaaS could be integrated into CD pipelines to resolve the aforementioned concerns (see Figure 1). API calls against SecaaS could automate security tasks, including security analysis and reports generation.

IV. SECURITY REQUIREMENTS FOR MIGRATION TO CNA AND CAVAS SYSTEM MODEL

We used the CAVAS (Figure 2) as the target application to investigate SecaaS migrations to CNA. CAVAS was originally a monolithic Dynamic Application Security Testing (DAST)

²<https://12factor.net>

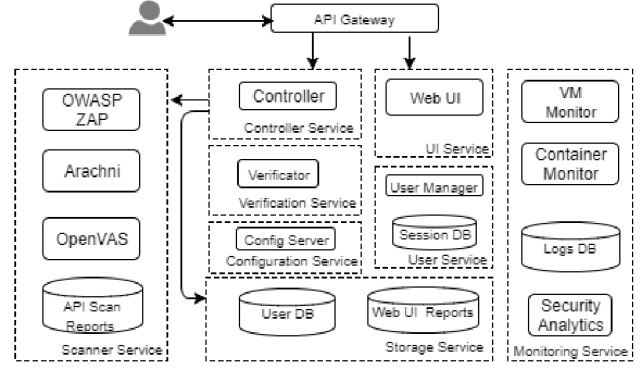


Figure 3: CAVAS Redesigned Using Cloud Native Design Patterns

application, for conducting security assessments. We developed it for previous research[22][23] on cloud vulnerability assessments. CAVAS was selected as the reference application for this work considering our existing knowledge and working understanding of the application. Familiarity with applications slated for migration increases chances of success given the huge code re-factoring procedure [24]. In the next subsections, we first discuss on the security requirements for CNA migrations and provide details of the steps taken during CAVAS migration. Thereafter, we describe the converted application's system model.

A. Security Requirements and Considerations

Some requirements for migration to CNA include:

1) *Application Security*: The process of decomposing an application into smaller, autonomous components often requires major code refactoring or even development of new services. Each microservice could be internally developed with different programming languages/frameworks. This diversity of technologies incurs additional overhead for security e.g. vulnerability assessments have to consider these disparate technologies as against limited technologies in monolithic applications.

2) *Network Security*: The security of the target migration environment requires security evaluation. This is essential, particularly on public cloud platforms, where the security is a shared responsibility between the Cloud Service Providers (CSP) and tenants. For example, security scanners can be used to scan target VMs for check for vulnerabilities prior to application deployment. Furthermore, firewalls, security groups and other cloud networking security measures ought to be configured beforehand. However, this security evaluation depends cloud's deployment model. The measures above are specific to IaaS, there might be no need for similar measures on PaaS [16].

3) *Data Security*: Data security is an important security requirement, the impact of data leakage or compromise during migration could severely affect business processes. Data is essentially in motion during migration and exposed to several

attack vectors such as MiTM attacks. A possible migration strategy for databases is a phased approach in which databases are migrated via multiple steps. There are two main database models employed in CNA, *shared database* model in which several microservices use a common database, and *one-database-per-service* model, in which each microservice instance maintains its own database [16]. The later model involves more complexities hence additional security effort is required. However, regardless of the approach, appropriate measures like data encryption could be considered to reduce security risks.

4) *Security Monitoring*: Application monitoring is already ingrained into cloud native design due to the *observability* pattern. Accordingly, several frameworks for implementing CNA integrate logs and metrics aggregation features. However, these logs and metrics are specifically designed for performance troubleshooting and routine health checks. Security monitoring is not available in most CNA implementations and frameworks. Security monitoring could aid in identifying security threats and securing the enlarged attack surfaces.

B. Justification for Migration to CNA and Redesign Efforts

Efficient auto-scaling was a requirement for CAVAS owing to the need to efficiently handle unpredictable surges in scan requests. A common solution for handling requests overload is to horizontally auto-scale the overloaded components [25] [26]. However, monolithic applications often scale vertically, are in-flexible and non-fault tolerant [16]. On the contrary, CNA, offer the possibility of horizontal scaling, per microservice. Also, security assessment operations are usually long running tasks and CPU-intensive, for example a comprehensive scan of a simple website could take 15 - 20 minutes. Hence, horizontal scaling is desirable for parallel processing of multiple requests. Similarly, a scale down is useful when there are few requests as this saves the costs i.e. only minimum number of cloud Virtual Machine (VM)s should be left active.

The redesign of CAVAS followed several steps and considerations. Firstly, a microservice approach was used to redesign and implement the new application. The components were decomposed as separate entities as shown in Figure 3. Next we applied cloud-focused design patterns such as *load-balancing*, *service discovery*, *externalized configuration* and *API gateway* [5], implementation details are at Section V.

For deployment strategy, three approaches were considered: 1) *containerized* environments such as Docker 2) cloud environments i.e. (PaaS or IaaS) 3) a hybrid of the former two[8]. We choose the hybrid approach i.e by employing an OpenStack cloud and Docker containers. OpenStack already implements several cloud native design patterns such as scalability and rapid elasticity. The choice for integrating containers was owing to the inherent performance and scalability afforded by container technologies. Also, monitoring non-OpenStack components when deployed in containers could be easier and more efficient than using OpenStack virtual instances. Note the changes made to the *data access layer*, the *Scanner Service* uses the *shared database strategy* [16],

for storing information e.g. scanner configurations. A Redis database is deployed in the *User Service* for retaining session information, externalising service state from the *UI Service* i.e. *statelessness*. This approach affords better flexibility to scale and easy *disposability* [27] i.e. an unhealthy service can be easily replaced with another one.

C. System Model

CAVAS is deployable in two modes : as an integral module of a development pipeline (Figure 1) or in a *manual testing* scenario i.e. via the web GUI. In the later mode, a user supplies a target resource's domain name or IP-address for security testing (Figure 3). The scan request is sent through the API Gateway to the UI Service, where the user is authenticated. The request is forwarded to the Verification Service, if target resource ownership verification is necessary. More details of the verification approaches we implement are at Section V-E. Following positive verification, the request is passed to the *Controller Service*, which makes an API call to the *Scanner Service*. Here, the request is further directed to the respective scanning engine based on the type of request i.e. requests for a web application tests are directed to either Arachni [28] or OWASP ZAP [29]. The scanning operation is launched, on completion the report is relayed to any instance of the UI Service, where it is provided for the user.

V. IMPLEMENTATION

This section provides implementation details of our cloud native SecaaS - CAVAS. We used Spring Cloud framework (Java) ³ to enable the core cloud native design patterns. Our implementation was deployed on a private OpenStack IaaS cloud with two nodes i.e. compute and storage node.

A. Cloud Native Orchestration

We orchestrated the Controller service, UI Service, Verification Service and User Service in separate VMs using *uber jars*, while the Scanner service and Monitoring Service are orchestrated in docker containers. The storage service is an OpenStack Database-as-a-Service (DBaaS) implementation combining Swift storage and Trove storage. The VM are provisioned using pre-installed cirros images. Cirros images are for testing and hence not suitable for our use-case. Therefore we created custom images using Ubuntu cloud images, ⁴ installed Java Development Kit (JDK), Apache Tomcat 7 web-server ⁵ and FileBeat plugin (monitoring agent). The images are thereafter translated to *image templates* using OpenStack Glance. Our custom image configuration is: *vCPU:2, RAM:2GB, HDD:40GB, flavor:m1.medium*.

B. Implementing Cloud Native Design Patterns

We implemented a subset of the cloud native design patterns, as required for our use case. Registry and Service Discovery is a pattern that enables deployed service instances

³<https://pivotal.io/cloud-native>

⁴<https://cloud-images.ubuntu.com>

⁵<https://tomcat.apache.org>

locate themselves. We implement this with the Spring's implementation of Netflix Eureka.⁶ All service instances are mandated to register specific metadata e.g. ip addresses and port numbers with the Eureka server. The Eureka server performs periodic health checks on registered services every 3 minutes, services are de-registered after three consecutive health check failures. CAVAS microservices are loose-coupled, with the advantage of scaling independently if requests breach the maximum threshold. Service configurations e.g. passwords and database connection information are externalized i.e. retained at the Configuration Service. This approach follows after the externalized configuration design pattern. Similarly, the API gateway pattern⁷ is implemented using Spring Cloud API gateway. The gateway receives all incoming requests and forwards them to the appropriate services as described in Section IV-C.

C. Scaling-Engine

Horizontal scaling is an important property of CNA. It affords automated increase or reduction of resources based on metrics specified in scaling policies. These metrics are collected in real-time by the monitoring service. A pool of UI service instances are auto-scaled when placed behind a load-balancer. Requests from the API gateway are routed through the loadbalancer, and distributed to the UI service instances using *round-robin algorithm*. Algorithm 1 implements the *scale-up policy*. Firstly, the list of VMs in the monitored network *vNet* and subnet *vSubNet* is obtained from the Nova service. Then a check is performed for CPU consumption i.e. if the *cpu_util* metric is more than 60% for more than 120 seconds (*rangeTime*). If yes, another instance is launched (using *cloud config*⁸) executed by line 7 of Algorithm 1, assuming the maximum number of instances *maxAllowed* is not exceeded. In our case the maximum number of allowed instances is 4. A UI service jar is fetched from the Swift storage and installed in the new VM. On completion of VM startup, and instance launches and registers with the Eureka server. For scaling down, a VM is shutdown if the *cpu_util* is 15% or less for atleast 30 seconds. The UI service instance is de-registered from the Eureka server and subsequent traffic is diverted to other UI service instances.

D. Security Testing

The actual security testing is done by the security scanners integrated into the Scanner Service. CAVAS tests web applications and networks hosts for security vulnerabilities using these scanners. On reception of a request from the API gateway, the UI service automatically generates appropriate scan profiles based on specified vulnerability vectors. Scanning profiles specify the depth of testing to be conducted, this could be basic or comprehensive. Requests are forwarded to the appropriate scanner i.e. network scans are sent to OpenVAS while web application test requests are sent to either Arachni

⁶<http://cloud.spring.io/spring-cloud-netflix/spring-cloud-netflix.html>

⁷<http://microservices.io/patterns/apigateway.html>

⁸<https://cloudinit.readthedocs.io/en/latest/index.html>

Algorithm 1 ScaleUp Algorithm

```

1: procedure SCALEUP(vNetid)
2:   Get list of virtual instances in vSubNet of vNet, vList
3:   for each vInstance in vList do
4:     if cpu_util > 60 percent then
5:       if rangeTime > 120 seconds then
6:         if vListSize < maxAllowed then
7:           start another vInstance
8:           deploy CAVAS_WebApp
9:           configure CeilometerAlarms
10:          update loadBalancer
11:         end if
12:         if vInstance isActive then
13:           launch MonitoringAgent
14:         end if
15:       end if
16:     end if
17:   end for
18: end procedure

```

or OWASP ZAP. On completion, scan results are retrieved, respective session information are also collected from Redis database and the result is sent to the original requester. All instances of the UI Service can identify the requester and deliver scan reports based on the retrieved session information.

E. Security Assessment Verification

Abuse and nefarious use of cloud services is one of the top threats against cloud services [30]. For example, a hacker could test a target-site using a SecaaS platform and subsequently attack the target-site based on revealed security weaknesses. Victims could wrongly accuse the SecaaS provider, as an investigation could show previous scans from the providers's endpoint. Hence, we use the verification techniques below to mitigate this threat. These techniques are limited to manual testing, API access secrets/keys are employed for CI integration (Figure 1).

1) *HTML Page Upload Technique*: The requester is mandated to upload a HTML file containing a verification code (*verifCode*) generated based on his request and profile. The *Verificator* then makes a *HTTP GET* request to validate the page and the *verifCode*.

2) *Meta-Tag Technique*: The meta-tag verification technique requires the requester to paste a uniquely generated meta-tag in the header of the target site. Thereafter, the *Verificator* crawls the page and validates the meta-tag and *verifCode* before granting the scanning request.

3) *DNS Look-Up Technique*: By performing DNS lookups queries against a target site, information about the registrar is retrieved and compared with requester's registered information. Thereafter, a verification email with an embedded *verifCode* is sent to the email address retrieved during the DNS lookup. Scanning request is granted following reception of a verification response (*verifEmailResp*) with the expected *verifCode*.

Table I: Summary of Evaluation Environment

Environment Variable	Value
Server Operating System	Ubuntu 14.04 Trusty Tahr
Cloud Environment	OpenStack Newton
Web Server	Apache Tomcat 7
RAM	32GB
HardDrive Capacity	500GB
Number of Users	50
Ramp Up Duration	500 seconds
Number of Processing Requests	50
Security Test Profile	.xss and csrf vulnerabilities
Duration for security test	20 minutes

F. Data Security

Given security assessments results are *sensitive* material [31], we securely encrypt and persist scan reports in the Swift storage with Swift’s native encryption [32]. This feature is not enabled by default, hence we enabled it. Reports are therefore automatically encrypted with AES-256 encryption algorithm in CTR mode. Decryption is automatically triggered by GET requests against specified objects. An internal key management strategy is employed by Swift to manage the *MasterSecretKey* used for encryption. Reports are directly accessible via the web interface using auto-generated *TempURLs*, a technique for securing transmitted data with cryptographic signatures and access validity periods.

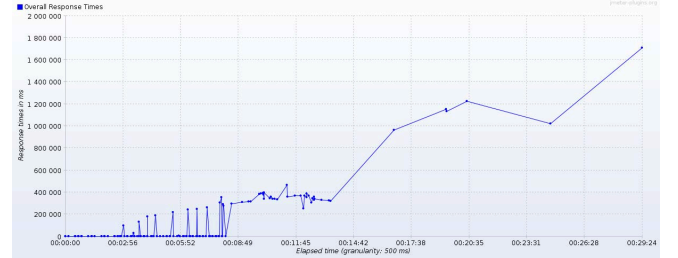
G. Cloud Monitoring

The *Monitoring Service* consists of three modules: *Compute Pool Monitor*, *Container Monitor* and *Security Analytics* modules. The VM Monitor and Container Monitor are responsible for monitoring services deployed in VMs and docker containers respectively. Rule-based metrics [33] are collected from the VMs using OpenStack Telemetry, which gathers comprehensive metrics from all OpenStack services. Aodh is the OpenStack alarming service, for triggering specific actions when metrics thresholds are breached. We leverage Elastic stack [34] in the VM Monitor. Firstly, logs are aggregated with the FileBeat plugin, which parses and ships logs to Logstash. Logstash further processes and forwards logs to Elasticsearch. Thereafter, logs can be queried and visualized via Kibana⁹, a web based visualization framework. We deployed Prometheus¹⁰ and cAdvisor¹¹ in the Container Monitor. cAdvisor aggregates metrics from containerized resources and physical hosts of the compute and storage nodes. The Security Analytics sub-service analyzes aggregated logs for anomaly events e.g consistent failed login attempts.

VI. EVALUATION

In this Section, we describe our experiments and analyze the results. *Scanner Time Analysis* and *Vulnerability Detection Analysis* [35] were conducted against two versions of CAVAS, a cloud-native version and a non cloud-native version. For

simplicity, we refer to both versions as *Version A* and *Version B* respectively. We deployed the OWASP Juice Shop¹², a deliberately vulnerable, modern web application as the test web application. We use Apache JMeter¹³, an open-source load testing framework to instrument loads i.e. user requests. We also present a simple case study to validate the effectiveness of our security assessment verification techniques.



(a) Performance of the Cloud Native Application (Version A)



(b) Performance of the non-Cloud Native Application (Version B)

Figure 4: Comparison of Performance Evaluation Results

A. Scanner Time Analysis

Table I is the summary of our test environment, we assume that we have 50 users using the platform for 500 seconds. Figure 4a and Figure 4b show the *Average Response Times* for *Version A* and *Version B* respectively. From these figures, we observe that *Version A* out-performs *Version B* with about 6 minutes. The difference in performance is attributed to *Version A*’s capacity to handle more traffic since the load is distributed to multiple web-servers. Accordingly, web requests are handled faster and more efficiently. For *Version B*, load is directed to a single web server which is limited in the number of concurrent requests it processes per time.

B. Vulnerability Detection Analysis

The vulnerability detection analysis was to measure vulnerability detection per time. We aimed at verifying if the earlier obtained scanner time analysis results correlate with vulnerability detection and to derive the marginal difference. Arachni [28] was used as the vulnerability scanner with a slightly modified version of the default *scan profile*. We instrumented 10 scanning requests within 30 seconds (simulating 10 concurrent users) for a period of 10 minutes. As shown in

⁹<https://www.elastic.co/products/kibana>

¹⁰<https://prometheus.io/>

¹¹<https://hub.docker.com/r/google/cadvisor>

¹²<https://github.com/bkimminich/juice-shop>

¹³<https://jmeter.apache.org/>

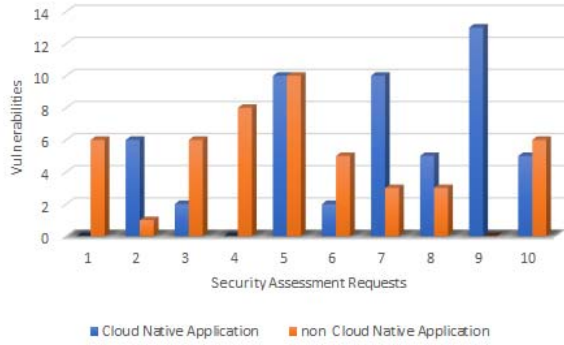


Figure 5: Security Analysis of the CNA and non-CNA Implementations

Figure 5, Versions A and B identified 53 and 48 vulnerabilities respectively. The performance results show that Version A outperforms Version B with about 11 percent. This correlates with the performance results above, in which *Version A* outperformed *Version B*. We also observe that no vulnerabilities are identified during the first and fourth requests of *Version A* and ninth requests of *Version B*. This could have been as a result of traffic congestion, and could be overcome by introducing request-based scaling.

C. Case Study: Security Assessment Verification Techniques

To demonstrate the effectiveness of the security assessment verification techniques (Section V-E), we present a fictitious case study analysis. Alice is the administrator of *CloudScan*, a SecaaS provider. Bob is a former employee of *CloudScan*, recently laid-off for disciplinary reasons. Bob is unhappy and wants to get back at *CloudScan* via a reputation smearing attack. Hence, he signs up for *CloudScan* service as Mark Pane, the Administrator of *DynamicStores*, an online retailer shop. He then proceeds to request for a comprehensive security assessment of *DynamicStores*. He plans to use the detailed, high quality results to attack *DynamicStores*. If successful, an incident response investigation could observe scan traces from *CloudScan*. During the request, he is presented with the site ownership verification techniques outlined in Section V-E. He selects the *DNS Lookup* option, *DynamicStores regInfo* is retrieved via *DNSLookUP*. Although the requester's name corresponds with the registrars's name, Bob is unable to verify the *verifEmail* request sent to the registered email address. Ultimately his request is not granted and after attempts at using the other methods fails, he gives up his entire plot.

VII. CONCLUSION AND FUTURE WORK

SecaaS applications provide high quality security services using the SaaS cloud delivery model. This approach to security prospectively affords robust security for enterprises given the provision of easy and cheap access to security without the need for complex installations and configurations. However, the current development and deployment mechanisms for SecaaS applications do not sufficiently scale to demands e.g.

unpredictable request peaks. This challenge is caused majorly by employment of architectural designs not specifically fitted for cloud platforms. We tackled this challenge by employing a new approach for cloud application development i.e. *cloud native design patterns*. Cloud native design patterns combine microservices with cloud focused design patterns thereby enabling cloud-native properties such as *externalized configuration* and *loose-coupling*. We also implement countermeasures against the security issues commonly introduced by these design patterns.

Future interesting research include automation of CNA migrations with appropriate pre-migration security evaluation. The possibility of leveraging orchestration engines e.g. Topology and Orchestration Specification for Cloud Applications (TOSCA) and AWS CloudFormation, with well defined security policies is also an interesting research focus.

REFERENCES

- [1] P. Wood, B. Nahorney, K. Chandrasekar, S. Wallace, and K. Haley, "Symantec global internet security threat report", *White Paper, Symantec Enterprise Security*, vol. 21, 2016.
- [2] IBM, *2017 ibm x-force threat intelligence index*, Accessed 13.08.2017. [Online]. Available: <https://assets.documentcloud.org/documents/3527813/IBM-XForce-Index-2017-FINAL.pdf>.
- [3] J. Cito, P. Leitner, T. Fritz, and H. C. Gall, "The making of cloud applications: An empirical study on software development for the cloud", in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, ACM, 2015, pp. 393–403.
- [4] X. Zhang, N. Wuwong, H. Li, and X. Zhang, "Information security risk management framework for the cloud computing environments", in *Computer and Information Technology (CIT), 2010 IEEE 10th International Conference on*, 2010.
- [5] A. Balalaie, A. Heydarnoori, and P. Jamshidi, "Migrating to cloud-native architectures using microservices: An experience report", in *European Conference on Service-Oriented and Cloud Computing*, Springer, 2015, pp. 201–215.
- [6] P. Jamshidi, "Microservices architecture enables devops",
- [7] N. Kratzke and P.-C. Quint, "Understanding cloud-native applications after 10 years of cloud computing—a systematic mapping study", *Journal of Systems and Software*, 2017.
- [8] N. Kratzke and R. Peinl, "Clouns—a cloud-native application reference model for enterprise architects", in *Enterprise Distributed Object Computing Workshop (EDOCW), 2016 IEEE 20th International*, IEEE, 2016, pp. 1–10.
- [9] K. Hoffman, *Beyond the twelve-factor app*. O'Reilly Media, Inc, 2016.

- [10] A. Furfaro, A. Garro, and A. Tundis, "Towards security as a service (secaas): On the modeling of security services for cloud computing", in *Security Technology (ICCST), 2014 International Carnahan Conference on*, IEEE, 2014, pp. 1–6.
- [11] K. Salah, J. M. A. Calero, S. Zeadally, S. Al-Mulla, and M. Alzaabi, "Using cloud computing to implement a security overlay network", *IEEE security & privacy*, vol. 11, no. 1, pp. 44–53, 2013.
- [12] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, today, and tomorrow", *ArXiv preprint arXiv:1606.04036*, 2016.
- [13] V. Varadharajan and U. Tupakula, "Security as a service model for cloud environment", *IEEE Transactions on Network and Service Management*, vol. 11, no. 1, pp. 60–75, 2014.
- [14] S. Brunner, M. Blöchliger, G. Toffetti, J. Spillner, and T. M. Bohnert, "Experimental evaluation of the cloud-native application design", in *Utility and Cloud Computing (UCC), 2015 IEEE/ACM 8th International Conference on*, IEEE, 2015, pp. 488–493.
- [15] A. Balalaie, A. Heydarnoori, and P. Jamshidi, "Microservices architecture enables devops: Migration to a cloud-native architecture", *IEEE Software*, vol. 33, no. 3, pp. 42–52, 2016.
- [16] F. Leymann, U. Breitenbücher, S. Wagner, and J. Wettinger, "Native cloud applications: Why monolithic virtualization is not their foundation", in *International Conference on Cloud Computing and Services Science*, Springer, 2016, pp. 16–40.
- [17] F. Leymann, C. Fehling, S. Wagner, and J. Wettinger, "Native cloud applications: Why virtual machines, images and containers miss", in *Proceedings of the 6th International Conference on Cloud Computing and*, SciTePress, pp. 7–15.
- [18] M. Stine, *Migrating to cloud-native application architectures*. O'Reilly Media, Inc.1005 Gravenstein Highway North, Sebastopol, CA 95472., 2015.
- [19] C. Fehling, F. Leymann, R. Retter, W. Schupeck, and P. Arbitter, "Cloud application architecture patterns", in *Cloud Computing Patterns*, Springer, 2014, pp. 151–238.
- [20] L. Bass, I. Weber, and L. Zhu, *Devops: A software architect's perspective*. Addison-Wesley Professional, 2015.
- [21] A. A. Ur Rahman and L. Williams, "Software security in devops: Synthesizing practitioners' perceptions and practices", in *Proceedings of the International Workshop on Continuous Software Evolution and Delivery*, ACM, 2016, pp. 70–76.
- [22] K. A. Torkura and C. Meinel, "Towards cloud-aware vulnerability assessments", in *Signal-Image Technology & Internet-Based Systems (SITIS), 2015 11th International Conference on*, IEEE, 2015, pp. 746–751.
- [23] k. A. Torkura and C. Meinel, "Towards vulnerability assessment as a service in openstack clouds", in *Local Computer Networks Workshops (LCN Workshops), 2016 IEEE 41st Conference on*, IEEE, 2016, pp. 1–8.
- [24] M. Ahmadvand and A. Ibrahim, "Requirements reconciliation for scalable and secure microservice (de) composition", in *Requirements Engineering Conference Workshops (REW), IEEE International*, IEEE, 2016, pp. 68–73.
- [25] M. Mao, J. Li, and M. Humphrey, "Cloud auto-scaling with deadline and budget constraints", in *Grid Computing (GRID), 2010 11th IEEE/ACM International Conference on*, IEEE, 2010, pp. 41–48.
- [26] T. Llorido-Botran, J. Miguel-Alonso, and J. A. Lozano, "A review of auto-scaling techniques for elastic applications in cloud environments", *Journal of Grid Computing*, vol. 12, no. 4, pp. 559–592, 2014.
- [27] G. Toffetti, S. Brunner, M. Blöchliger, F. Dudouet, and A. Edmonds, "An architecture for self-managing microservices", in *Proceedings of the 1st International Workshop on Automated Incident Management in Cloud*, ACM, 2015, pp. 19–24.
- [28] T. Laskos, *Arachni web application security scanner framework*, 07.03.2017, 2017. [Online]. Available: <http://www.arachni-scanner.com/>.
- [29] OWASP, *Owasp zed attack proxy*, Accessed 04.02.2017, 2017. [Online]. Available: https://www.owasp.org/index.php/OWASP_Zed_Attack_Proxy_Project.
- [30] C. S. Alliance, *The treacherous 12: Cloud computing top threats*, Accessed 31.07.2017, 2016. [Online]. Available: https://downloads.cloudsecurityalliance.org/assets/research/top-threats/Treacherous-12-Cloud-Computing_Top-Threats.pdf.
- [31] C. S. alliance, *Secaas implementation guidance. category 5, security assessments*. [Online]. Available: https://downloads.cloudsecurityalliance.org/initiatives/secaas/SecaaS_Cat_5_Security_Assessments_Implementation_Guidance.pdf.
- [32] O. Foundation, *Openstack swift encryption*, Accessed 05.05.2017. [Online]. Available: https://docs.openstack.org/developer/swift/overview_encryption.html.
- [33] T. Llorido-Bostrán, J. Miguel-Alonso, and J. A. Lozano, "Auto-scaling techniques for elastic applications in cloud environments", *Department of Computer Architecture and Technology, University of Basque Country, Tech. Rep. EHU-KAT-IK-09-12*, 2012.
- [34] Elastic, *Kibana*, Accessed 01.04.2017. [Online]. Available: <https://www.elastic.co/products/kibana>.
- [35] J. Bau, E. Bursztein, D. Gupta, and J. Mitchell, "State of the art: Automated black-box web application vulnerability testing", in *Security and Privacy (SP), 2010 IEEE Symposium on*, IEEE, 2010, pp. 332–345.